BACKEND ARCHITECTURE & API INTEGRATION SPEC

Extra Work / Additional Charges Flow

Change Order Specification: Provider Declaration, Customer Approval, and Checkout Integration

Target: **Backend Team & API Developers** Scope: **Orders Model • APIs • FCM • Payment Core** Priority: **High / Core Flow**

01 System Overview & Workflow Lifecycle

When a Service Provider completes an order, they can optionally declare extra work, spare parts, or additional charges. The Provider submits the extra amount and reason. The charge starts with `pending` status awaiting Customer decision:

Customer Accepts Charge
- Final Payable = **Original Amount + Extra Amount**
- Customer continues through existing checkout & payment flow.

Customer Rejects Charge
- Extra charge is dismissed (excluded from billing).
- Final Payable remains **Original Amount**.

⚠️ **MANDATORY ARCHITECTURE RULE: INSPECT CURRENT CODEBASE FIRST**

Existing code must be preserved. Do not blindly replace, rename, or duplicate existing functionality. Extend the existing architecture:

- **Preserve existing components:** Current `Order` model, database schema, status logic, `update-order-status` API, payment/checkout calculations, and FCM notifications.
- **No duplicate calculation logic:** Platform fee, gateway fee, VAT, commission, and other rules must not be duplicated or changed except to apply accepted extra amount.
- **Field & Key immutability:** Never rename existing DB columns or existing API response keys. Add only the specified new keys.
- **Single table schema:** Do not create new models/tables if the existing `orders` table can safely store these attributes.

02 Database Schema & Order Model Attributes

Add the following columns to the `orders` table only if they do not already exist:

Field / Column	Data Type	Default	Description & Business Rules
`extra_amount` FIXED	`decimal(10,2)`	`0.00`	Additional amount requested by Provider. Must be non-negative.
`extra_amount_reason` FIXED	`text / nullable`	`NULL`	Description/reason for extra work or spare parts.
`extra_amount_status` FIXED	`enum / string`	`'none'`	Approval lifecycle: `none`, `pending`, `accepted`, `rejected`.
`total_amount` FIXED	`decimal(10,2)`	*Existing total*	Final payable total (Original + accepted Extra Amount).

Allowed Values for `extra_amount_status`

NONE
No extra charge requested. Standard flow.

PENDING
Charge declared; awaiting Customer decision.

ACCEPTED
Customer approved. Extra added to payable total.

REJECTED
Customer rejected. Extra dropped from total.

03 Provider Completion API (Extend Existing)

`POST /api/update-order-status/{order_id}`

Rule: Do not create a duplicate endpoint. Extend the existing status update implementation.

PAYLOAD A: EXTRA WORK DECLARED

```
{
  "id": "123",
  "status": "completed",
  "extra_amount": "50.00",
  "extra_amount_reason": "Replaced valve & fitting"
}
```

Backend: Save values, set `extra_amount_status = 'pending'`, notify Customer.

PAYLOAD B: NO EXTRA WORK (STANDARD)

```
{
  "id": "123",
  "status": "completed",
  "extra_amount": "0.00",
  "extra_amount_reason": ""
}
```

Backend: Set `extra_amount_status = 'none'`. Standard completion logic continues unchanged.

EXPECTED API RESPONSE (EXTEND EXISTING STRUCTURE — KEEP ALL EXISTING KEYS)

```
{
  "status": true,
  "message": "Order completed successfully",
  "data": {
    "id": 123,
    "status": "completed",
    "price": "150.00",
    "extra_amount": "50.00",
    "extra_amount_reason": "Replaced valve & fitting",
    "extra_amount_status": "pending",
    "total_amount": "200.00"
  }
}
```

04 Order Details & Home API Integration

Return the 4 exact keys alongside the standard Order resource across Order Details & Home APIs:

```
{
  "id": 123,
  "status": "completed",
  "price": "150.00",
  "extra_amount": "50.00",
  "extra_amount_reason": "Replaced damaged water valve",
  "extra_amount_status": "pending",
  "total_amount": "200.00"
}
```

05 Customer Extra Amount Decision API (New Endpoint)

`POST /api/orders/{order_id}/extra-amount-action`

Requires Customer Bearer Token. Dedicated endpoint for Customer approval or rejection of requested extra charges.

REQUEST: ACCEPT EXTRA CHARGE

```
{
  "order_id": "123",
  "action": "accept"
}
```

- Status → **accepted**
- Total = Original (150) + Extra (50)
- **Payable: SAR 200.00** (proceed to checkout)

REQUEST: REJECT EXTRA CHARGE

```
{
  "order_id": "123",
  "action": "reject",
  "rejection_reason": "Did not authorize replacement"
}
```

- Status → **rejected**
- Extra charge dismissed from invoice
- **Payable: SAR 150.00**

SUCCESS RESPONSE STRUCTURE

```
{
  "order_id": 123,
  "extra_amount_status": "accepted",
  "original_amount": "150.00",
  "extra_amount": "50.00",
  "final_total": "200.00"
}
```

06 FCM Push Notifications (Customer & Provider)

Reuse existing FCM notification service. Notification `data.type` keys are fixed and must not be altered:

FCM 1: PROVIDER DECLARES EXTRA → CUSTOMER

```
{
  "notification": {
    "title": "Extra Work Approval Request",
    "body": "Provider requested SAR 50.00 for additional work."
  },
  "data": {
    "type": "EXTRA_PAYMENT_REQUEST",
    "order_id": "123",
    "extra_amount": "50.00",
    "extra_amount_reason": "Replaced damaged water valve"
  }
}
```

FCM 2: CUSTOMER DECIDES → PROVIDER

```
{
  "notification": {
    "title": "Extra Charge Decision",
    "body": "Customer has accepted your extra charge of SAR
  },
  "data": {
    "type": "EXTRA_PAYMENT_RESPONSE",
    "order_id": "123",
    "decision": "accepted"
  }
}
```

07 Validation & Settlement Business Rules

VALIDATION & GUARDS

- `extra_amount` : numeric, `min:0` , optional when 0.
- `extra_amount_reason` : required if `extra_amount > 0` .
- `action` : required, strict enum: `accept` or `reject` .
- **Status Guard:** Decision allowed *only* if status is `pending` .
- **Ownership Check:** Customer owns order & Provider is assigned.
- **Idempotency:** Disallow repeated processing of settled decisions.

PAYMENT ENGINE RULES

- **Accepted:** `Final Order Total = Original Amount + Extra Amount`
- **Rejected:** `Final Order Total = Original Amount`
- **Fee Calculations:** Recalculate VAT, App Fee, Payment Gateway Fee, and Provider Commission via existing payment services.
- **No Duplicate Logic:** Do not create separate fee calculators.

08 Implementation Checklist & Verification

BACKEND DELIVERABLES

- ☐ Inspect existing Order model, table migration, and total calculation
- ☐ Add migration for: `extra_amount` , `extra_amount_reason` , `extra_amount_status`
- ☐ Update Order model `$fillable` and attribute casting
- ☐ Extend `POST /api/update-order-status/{id}` to store extra charges
- ☐ Implement `POST /api/orders/{id}/extra-amount-action`
- ☐ Add status guard & idempotency check (reject if not pending)
- ☐ Dispatch `EXTRA_PAYMENT_REQUEST` FCM to Customer
- ☐ Dispatch `EXTRA_PAYMENT_RESPONSE` FCM to Provider
- ☐ Update Order Detail & Home API transformers to output 4 fixed keys
- ☐ Ensure payment settlement services accurately compute final total

MOBILE APP ALIGNMENT CHECK

- ☐ Provider: Add optional extra amount & reason inputs to Complete sheet
- ☐ Customer: Display approval bottom sheet on order tracking screen
- ☐ Show itemized breakdown: Original Price + Extra Work = Final Payable
- ☐ Connect Accept & Pay Extra and Reject Extra buttons to new API
- ☐ Listen for FCM `EXTRA_PAYMENT_REQUEST` to prompt Customer UI
- ☐ Listen for FCM `EXTRA_PAYMENT_RESPONSE` to notify Provider
- ☐ Reload order status after decision without full screen restart
- ☐ Direct Customer into standard checkout upon accepting extra work